```
  destIndex[pixel.index].color_index ← sampled_color.index
#5
```

#1 Create an S-tree starting from the list of 256 sampled colors – here we explicitly pass the dimension of the search space, 3, just for clarity (it can be inferred from the points in the list, of course). We assume that each of the sampled colors, in the container, will be associated with its index in the destination color scale.
#2 Go through each pixel in the source image.
#3 In the most generic case pixels could be modeled as objects with more fields than just color, for instance their position in the image. If that's so, we just need the RGB components for the NN search.
#4 Find the nearest neighbor of current color.
#5 In the destination image, set the color index for the transformed pixel based on the index in the destination color scale of the sampled color that is closest to the original pixel's color.

For each pixel in the original image, we search its nearest neighbor in the tree, and store the index of the color in the destination scale closest to the original color of the pixel. Listing 11.9 shows pseudocode for these operations, using an S-tree: you can see how simple it is to perform such advanced tasks, when you have the right data structure! That's because all the complexity is encapsulated in the implementation of the S-tree (or k-d tree or equivalent), and that's also why the main goal of this book is helping readers to recognize the situations where they can use these data structures: that alone will make you stand as a developer, producing faster and more reliable code.